

Table des matières

1. Pour commencer
2. Structures et chaînes de caractères
3. Table d'adresses et de vide
4. Éditeur de niveaux pour le projet
5. Sprites et couleurs

1. Pour commencer

Le but de ce TP est de tester la lecture/écriture de chaînes de caractères et de structures dans les fichiers binaires. Les deux dernières parties présentent l'éditeur qui fait partie du projet.

Même si vous ne terminez tous les exercices lors de cette séance, il est fortement conseillé de le faire **avant** le TP prochain. De plus, commencez dès que possible l'éditeur qui ne comptera que pour une petite partie du projet, mais qui peut vous bloquer pour la suite si vous ne l'avez pas terminé.

2. Structures et chaînes de caractères

Dans cet exercice, nous allons tester la sauvegarde et la lecture de structures et de chaînes de caractères dans un fichier binaire.

1. Écrivez un programme qui prend en argument le nom d'un fichier. S'il n'existe pas, un fichier vide est créé. Le programme demande ensuite à l'utilisateur de saisir une chaîne de caractères. Elle est ajoutée à la fin du fichier. Ensuite, toutes les chaînes de caractères du fichier sont affichées, une par une. La dernière doit correspondre à la chaîne de caractères qui a été ajoutée en dernier.

Vous devez lire les chaînes de caractères une par une et non les lire toutes en une seule fois.

2. Créez une structure `personne_t` contenant un nom et un prénom (une chaîne de taille fixe de 20 caractères), ainsi qu'un âge (un entier). Initialisez la structure de manière statique (directement dans votre code) puis sauvegardez-la dans un fichier. Vérifiez son contenu avec `od`.
3. Maintenant, écrivez un programme qui permet de lire la structure depuis le fichier.
4. Modifiez vos deux programmes précédents en utilisant des `char*` pour le nom et le prénom.
5. Vérifiez le contenu du fichier avec `od`.

3. Table d'adresses et de vide

Nous souhaitons mettre en place une architecture dans un fichier binaire qui se base sur les tables d'adresses des systèmes de fichiers. Ici, les adresses sont des positions (off_t) dans le fichier. Pour cet exercice, nous partons sur un tampon de taille variable qui peut être représenté comme une chaîne de caractères. La table d'adresses possède N adresses et est sauvegardée au début du fichier. Elle permet d'accéder directement à chaque chaîne de caractères. Si nécessaire, une autre table est créée à une position quelconque dans le fichier. Pour gérer les suppressions, nous avons également besoin d'une table de vide qui indique les trous dans le fichier et leur taille.

1. Proposez une structure pour la table d'adresses et la table de vide.
2. Écrivez les fonctions pour créer, supprimer, lire, écrire et afficher une table, l'ajout ou la suppression d'une entrée.

Pour tester vos structures/fonctions, nous souhaitons écrire plusieurs programmes :

- Le premier affiche toutes les tables et les chaînes de caractères, en précisant leur numéro.
- Le second prend une chaîne en argument et l'ajoute dans le fichier.
- Le troisième prend un numéro et supprime l'entrée correspondante.

Prenez soin à mettre uniquement une seule table d'adresses et une table de vide en mémoire au même moment. Vous ne devez pas charger tout le fichier intégralement en mémoire.

3. Écrivez les programmes demandés.

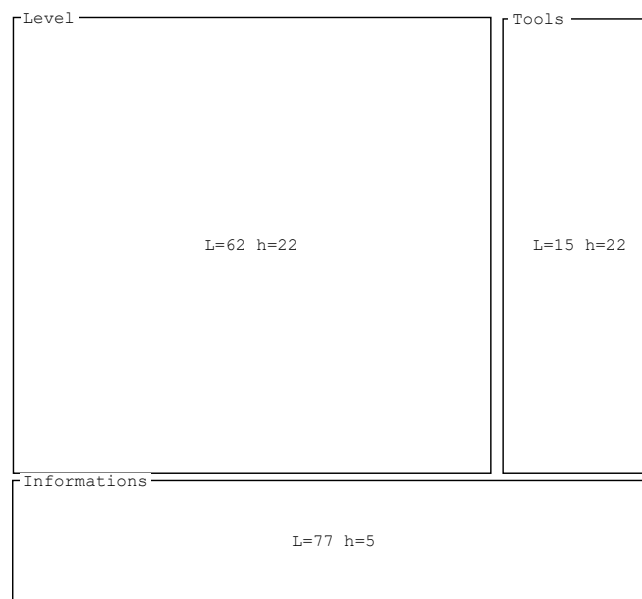
4. Éditeur de niveaux pour le projet

Le but de cet exercice est de présenter la première partie du projet final. Il s'agit de créer un éditeur de niveaux en exploitant l'interface *ncurses* présentée dans le TP précédent et en y ajoutant la gestion des fichiers.

Le monde est formé de plusieurs tableaux, chacun ayant une taille de 60 de large et 20 de haut et contient un ensemble d'éléments :

- Les blocs (*block*) : des éléments qui bloquent le passage ou qui empêchent de tomber.
- Des échelles (*ladder*) : elles permettent au joueur de monter/descendre.
- Des trappes (*trap*) : comme un sol/mur, sauf que ces blocs apparaissent/disparaissent à intervalles réguliers.
- Des clés (*key*) et des portails (*gate*) de 4 couleurs différentes : lorsque le joueur possède une clé donnée, il peut passer par le portail de la même couleur, sinon il est bloqué. Les clés sont placées dans les différents tableaux du monde.
- Des portes (*door*) : elles sont numérotées de 1 à 99 et fonctionnent par paire. Lorsque le personnage entre par la première, il ressort par l'autre.
- La sortie (*exit*) est la porte qu'il faut franchir en premier pour gagner.
- L'entrée (*start*) est la porte depuis laquelle le joueur arrive dans le monde (ou réapparaît quand il meurt).
- Des robots (*robot*) : ils se déplacent (vers la droite ou la gauche) dans le tableau ; s'ils touchent le joueur, celui-ci meurt.
- Des sondes (*probe*) : elles fonctionnent comme les robots sauf qu'elles volent (elles se déplacent dans toutes les directions).
- Des coeurs (*heart*) : permet de recharger les coeurs du personnage (5 max.).
- Des bombes (*bomb*) : elles permettent de recharger les bombes du joueur (max. 3). Celui-ci peut en poser une pour paralyser ses ennemis (ce qui lui permet de passer au travers les sondes et les robots) ou ses adversaires.

L'interface est constituée de trois fenêtres. La fenêtre "Level" permet d'afficher et de modifier le tableau actuel lorsque l'utilisateur clique sur une case, en fonction de l'outil sélectionné. La fenêtre "Tools" permet de sélectionner l'outil en cours. La fenêtre "Informations" permet d'afficher des informations pour l'utilisateur. L'image ci-dessous montre les dimensions des fenêtres et leur disposition (à respecter scrupuleusement) :



Voici un exemple d'édition :



Les flèches à côté du numéro de niveau permettent de naviguer entre les niveaux. Le bouton "DELETE" permet de supprimer le niveau courant (on vide le niveau et on ne garde que les bords).

Les niveaux d'un monde sont contenus dans un seul fichier binaire qui n'est pas mis en mémoire intégralement : seul le niveau en cours d'édition est chargé. Au début du fichier se trouve une table de positions permettant de connaître la position dans le fichier de chaque niveau (et les informations que vous jugerez utiles). Elle comporte au maximum 10 entrées. Si nécessaire, une nouvelle table doit être créée dans le fichier (la position de la prochaine table est donc précisée dans la table). Vous aurez également besoin d'une table de vide pour gérer les suppressions.

Avant d'écrire l'interface, commencez déjà par réfléchir sur les structures nécessaires et comment les enregistrer dans le fichier. Ensuite, créez les structures et toutes les fonctions utiles (création, sauvegarde, chargement, suppression, affichage, etc.). Testez toutes les fonctions sans utiliser *ncurses*. Enfin, une fois que tout est vérifié, vous pouvez lier l'éditeur avec les structures.

5. Sprites et couleurs

Voici les différents sprites et couleurs utilisés pour le projet :

block (bloc) :  (1x1)
caractère : ' ' (espace)
couleur = CYAN

trap (piège) :  (1x1)
caractère : '#' (hashtag)
couleur = CYAN

life (vie) :  (1x1)
caractère : 'V' ('V' majuscule)
couleur = RED

bomb (bombe) :  (1x1)
caractère 'o' (petit 'O')
couleur = WHITE

ladder (échelle) :  (3x1)
caractères : ACS_LTEE, ACS_HLINE, ACS_RTEE
couleur = YELLOW

start (début) :  (3x4)
caractères : ' ' (espace)
couleur = FD_MAGENTA

exit (sortie) :  (3x4)
caractères : ' ' (espace)
couleur = FD_YELLOW

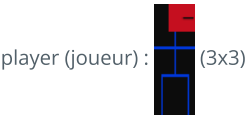
door (porte) :  (3x4)
caractères : ' ' (espace)
couleur = FD_GREEN

gate (portail) :  (1x1 => ici, 1x4, mais ce n'est pas obligatoire)
caractère : ACS_PLUS
couleur : MAGENTA, GREEN, YELLOW, BLUE (suivant le portail)

key (clé) :  (1x2)
caractères : ' ' (espace) et ACS_LLCORNER
couleurs : FD_MAGENTA (le haut) et MAGENTA (le bas) ; mais aussi FD_GREEN, GREEN, FD_YELLOW, YELLOW, FD_BLUE et BLUE (suivant la clé du portail)

probe (sonde) :  (3x2)
caractères : [ACS_LTEE, ACS_HLINE, ACS_RTEE], [ACS_LLCORNER, ACS_HLINE, ACS_LRCORNER]
couleur : WHITE

robot (robot) :  (3x4)
caractères : [ACS_ULCORNER, ACS_BTEE, ACS_URCORNER], [ACS_LLCORNER, ACS_TTEE, ACS_LRCORNER], [ACS_HLINE, ACS_PLUS, ACS_HLINE], [ACS_ULCORNER, ACS_BTEE, ACS_URCORNER]



caractères : tête [' ' (espace), '-' (tiret du 6)] ou l'inverse suivant l'orientation

couleur : FD_RED

caractères corps : [ACS_HLINE, ACS_PLUS, ACS_HLINE], [ACS_ULCORNER, ACS_BTEE, ACS_URCORNER], [ACS_VLINE, rien, ACS_VLINE]

couleur : BLUE, GREEN, YELLOW, RED (suivant le joueur)